Git

Git is a Distributed version control system

- It tracks changes in the source code
- Github is a cloud platform to store our source code
- Inode busting ?
- git cat-file -p db5e ?
  porcelain(high level) and plumbing(low level)
- `git config list` - to list the configurations like the username , email, ,editorr
- `git config --global init.defaultBranch master` - set the default branch name
- **Repository** - It is essentially a directory that contains a project . The only difference is that it contains a `.git` directory inside it . That is where Git stores all of its internal tracking and versioning informations for the project
- `git init` - iniitalises the directory with git
- `git log` - shows the commit log of the repo

Status

A file can be in one of the several states in a git repo like

- untracked - Not being tracked by git
- staged - marked for inclusion in the next commit
- committed - saved to the repositor's history
  `git status` shows the current status of the repo

Staging

without staging evey files in the repo will be inlcuded in every commit

- `git add .` - stages all the files in the current directory
- `git add file1 file2` - stages the files file1 and and file2

Commit

commit is a snapshot of the repo at a given point of time . it is a way to save the state of the repo , and its how git tracks of changes in the project

- A commit comes with a message that describes the changes made in that commit
- `git commit -m 'commit message '` - make a commit with the provided message

Log

A git log is a list of commits, where each commit represents the full state of the repo at a given point of time

- it shows the history of the commits in the repo
- `git --no-pager log -n 10` shows the only last 10 commits in the log
- `git --no-pager log -n 10 --parents --graph` - ?
- `git log -p` - log with the changelogs

commit hashes

commit hashes are not only derived from the content changes but also depend on the

- commit message
- the authors name and email
- the date and time
- parent commit hashes

Trees and Blobes

- tree - gits way of storing a directory
- blob - gits way of storing a file

Storing data

git stores an entire snapshot of files on a commit per level

- to make ti efficient git compresses and packs files
- git deduplicate files that are the same across different commits

Config

- `git config --add --global user.name "username"` set the gloabl username
- `git config --add --global user.emal "email"` set the global email
- `git config --list --local` list the local configuration for a repo
- `git config --add --local webflyx.valuation "mid"` - set the valuation of the webflyx section to mid
- `git config --get webflyx.valuation` get the value of the field valuation in teh section webflyx (get is the default operation we can use this too instead `git config webflyx.valuation`)
- `git config --unset webflyx.cto` unset the field cto in the section webflyx
- `--unset-all` to purge all instance of a key from the configuration
- `git config --remove-section webflyx` - remove a section from the configuration configuration priority work tree > local > global > system

Branching

git branch allows us to track of different changes seperately

- branch is just a pointer to the commit
- `git branch` - to check which branch we are in currently and list all the branches of the current repo
- `git switch -c branchname` - to create a branch and switch it to the created branch
- `git branch -m master main` rename the branch master with the name main
- `git branch branchname` - creates a new branch
- `git switch branchname`, `git checkout branchname` - swith to a different branch (first one is the latest one)
- `git log --graph --all` - to show the all branches in a graph like way
- `git branch -d branchname` - to delete a branch

Merge

- f**ast forware merge** - mergin when the tip of the branch is same as the merge base
-

Rebase

rebase helps to take the diverging commits from one branch and move them to the tip of the branch

- it enables us to do faste forware merge
  A-B-C

  D-E
  A-B-C

  D-E

Key Differences Between `git pull` and `git fetch`

- **`git fetch`:** This command downloads changes from the remote repository to your local repository's remote-tracking branches. It does not update your working directory or merge any changes, allowing you to review updates before integrating them into your current branch[1][2].
  - Updates only the remote-tracking branches.
  - Does not alter the current branch or working directory, thus avoiding potential merge conflicts[1][2].
  - Allows for manual review of changes before merging, providing greater control over the integration process[1][2].
- **`git pull`:** This command is essentially a combination of `git fetch` followed by `git merge`. It retrieves changes from the remote repository and automatically merges them into your current branch, updating your working directory accordingly[1][2].

- Directly updates the current branch with the latest changes from the remote branch.
- Can lead to merge conflicts if there are discrepancies between local and remote changes[1][2].
- Provides a quicker way to synchronize with the remote repository but at the risk of unintended merges[1][2].

Differences Between `git merge` and `git rebase`

- **`git merge`:**
  - Combines the histories of two branches by creating a new "merge commit" that includes all changes from both branches.
  - Preserves the chronological order of commits, resulting in a non-linear history that reflects how development occurred over time.
  - Results in a merge commit that represents the integration of two branches.
  - Maintains a complete history of how changes were made, making it clear where each commit originated.
- **`git rebase`:**
  - Moves or "reapplies" commits from one branch onto another base branch, effectively rewriting the commit history.
  - Creates a linear history by placing the commits from the feature branch on top of the target branch's latest commit, which can simplify the project's history.
    - Does not create a merge commit; instead, it rewrites commit hashes as it moves commits to the new base.
  - The history appears cleaner and more linear, but it can obscure the original context of when and how changes were made.

** Conflict Resolution**

- **`git merge`:**
  - Presents conflicts all at once if there are any discrepancies between the branches being merged.
  - Typically easier for teams to manage since it preserves the entire context of both branches.
- **`git rebase`:**
  - Presents conflicts one commit at a time as each commit is reapplied, which can make resolving conflicts more manageable but potentially more tedious if there are many conflicts.

Detached HEAD

- **What It Means:** When you are in a detached HEAD state, you are no longer working on the tip of any branch. Instead, HEAD points directly to a commit, meaning that any new commits you create will not be associated with any branch. This can make them harder to find later unless you take specific actions to preserve them

- **How You Get There:** Common ways to enter a detached HEAD state include:
  - Running `git checkout` on a specific commit hash (e.g., `git checkout 8fd3350`).
  - Checking out tags or remote branches that are not currently tracked by local branches

Implications of Detached HEAD

- **Making Changes:** While in this state, you can still make changes and create commits. However, these commits will not belong to any branch, making it easy to lose them if you switch back to another branch without creating a new one[1][2].
- **Temporary Work:** The detached HEAD state is useful for:
  - Exploring past commits without affecting the current branch.
  - Testing experimental changes without committing them to the main project history.
  - Debugging issues using commands like `git bisect`[3].

Revert a change pushed to a remote repo

1. Using `git revert`

This is the safest method as it creates a new commit that undoes the changes from the specified commit without altering the commit history.

- **Steps:**
  1. Identify the commit hash you want to revert using:

     ```
     git log --oneline
     ```

  2. Run the revert command:

     ```
     git revert <commit_hash> --no-edit
     ```

     The `--no-edit` option prevents Git from opening an editor for a commit message.

  3. Push the changes to the remote repository:

     ```
     git push origin <branch_name>
     ```

2. Using `git reset` and Force Push

This method rewrites history by removing the commit from the branch, which can be problematic if others have based work on that commit.

- **Steps:**
  1. Reset your local branch to the previous commit:

     ```
     git reset HEAD^ --hard
     ```

     This command removes the last commit and all changes associated with it.

  2. Force push the changes to the remote repository:

```
git push origin +HEAD
```

The `+` indicates a forced non-fast-forward push, which updates the remote branch to match your local branch.

Differences Between Fast-Forward Merge and Recursive Merge

- **Fast-Forward Merge:**
  - A fast-forward merge occurs when the branch being merged has no additional commits compared to the branch you are merging into. In this case, Git simply moves the pointer of the target branch forward to the latest commit of the source branch, effectively avoiding the creation of a merge commit
- **Recursive Merge:**
  - A recursive merge is used when the branches have diverged, meaning there are commits on both branches since they split. This type of merge creates a new commit that combines the changes from both branches. It is the default strategy for merging in Git when there are conflicting changes

Purpose of `git cherry-pick`

1. **Selective Integration of Changes:**
   - Cherry-picking enables developers to select specific commits from one branch and apply them to another without merging the entire branch. This is particularly useful in scenarios where only certain changes are needed, such as bug fixes or feature enhancements.
2. **Correcting Mistakes:**
   - If a commit was made to the wrong branch, cherry-picking allows you to easily transfer that commit to the correct branch without needing to merge all changes from the incorrect branch.
3. **Backporting Changes:**
   - Cherry-picking is often used to backport bug fixes or features from a main development branch (like `main` or `develop`) to older release branches, ensuring that critical updates are applied where needed.
4. **Collaborative Development:**
   - In team environments, cherry-picking can be employed to incorporate changes proposed by other team members without merging their entire branch, allowing for more controlled integration of code.

How to Use `git cherry-pick`

- To cherry-pick a commit, you first need the commit hash (SHA) of the commit you want to apply. You can find this using:

```
git log --oneline
```

- Once you have the commit hash, switch to the target branch where you want to apply the commit:

  ```
  git checkout <target-branch>
  ```

- Then, execute the cherry-pick command:

  ```
  git cherry-pick <commit-hash>
  ```